

FORMAN CHRISTIAN COLLEGE

(A CHARTERED UNIVERSITY)



Course Code: COMP451

Session: SP26

Assignment Number: 3

Title of Lab/Assignment:

Construction of LR(0) Parser

Group Member/s Names and Roll Number/s:

Musfira Sehar, 280048443

Table of Contents

- Introduction
- Objective
- Grammar Augmentation
- LR(0) Items Construction
- DFA Construction (LR(0) Automaton)
- LR(0) Parsing Table
- Conflict Analysis
- Implementation Methodology
- Program Code
- Test Cases
- Parsing Output Format
- Conclusion

1. Introduction

LR(0) parsing is a bottom-up parsing technique used in compiler design. It constructs a deterministic finite automaton (DFA) from grammar productions and uses a stack-based parsing approach.

LR(0) parsers do not use lookahead symbols and are the simplest form of LR parsers.

2. Objective

The objectives of this assignment are:

- To construct LR(0) items for a grammar
 - To build canonical collection of item sets
 - To construct DFA for LR(0) items
 - To build ACTION and GOTO parsing tables
 - To implement LR(0) parser in C
 - To validate input strings using stack-based parsing
-

3. Grammar Augmentation

Given Grammar:

$S \rightarrow AB$
 $A \rightarrow aA \mid b$
 $B \rightarrow c$

Augmented Grammar:

$S' \rightarrow S$
 $S \rightarrow AB$
 $A \rightarrow aA \mid b$
 $B \rightarrow c$

4. LR(0) Items Construction

Item Sets:

I0:

$S' \rightarrow \bullet S$
 $S \rightarrow \bullet AB$
 $A \rightarrow \bullet aA$
 $A \rightarrow \bullet b$
 $B \rightarrow \bullet c$

I1:

$S' \rightarrow S \cdot$

I2:

$S \rightarrow A \cdot B$

$B \rightarrow \cdot c$

I3:

$A \rightarrow a \cdot A$

$A \rightarrow \cdot a A$

$A \rightarrow \cdot b$

I4:

$A \rightarrow b \cdot$

I5:

$B \rightarrow c \cdot$

I6:

$S \rightarrow AB \cdot$

5. DFA Construction

(LR(0) Automaton)

States represent item sets:

- $I_0 \rightarrow$ initial state
 - $I_1 \rightarrow$ accept state
 - Transitions:
 - $I_0 \xrightarrow{S} I_1$
 - $I_0 \xrightarrow{A} I_2$
 - $I_0 \xrightarrow{a} I_3$
 - $I_0 \xrightarrow{b} I_4$
 - $I_2 \xrightarrow{B} I_6$
 - $I_2 \xrightarrow{c} I_5$
-

6. LR(0) Parsing Table

State	a	b	c	\$	S	A	B
0	S3	S4			1	2	
1				ACC			
2			S5				6
3	S3	S4				7	
4	R A→b						
5	R B→c						
6	R S→AB						

7. Conflict Analysis

Result:

- No shift-reduce conflicts observed in this grammar
- No reduce-reduce conflicts observed

Conclusion:

The grammar is **LR(0) compatible**.

If conflicts existed, they would occur due to:

- Ambiguity in grammar
 - Multiple reductions in same state
 - Shift vs reduce decision overlap
-

8. Implementation Methodology

The LR(0) parser works as follows:

1. Augment grammar
2. Generate LR(0) items

3. Construct closure() and goto() functions
 4. Build canonical collection of item sets
 5. Construct DFA transitions
 6. Build ACTION and GOTO tables
 7. Use stack-based parsing algorithm
 8. Read input string and process step-by-step
 9. Accept or reject based on final state
-

9. Program Code

(C Implementation)

```
#include <stdio.h>
#include <string.h>

char stack[100];
char input[100];
int top = -1;
int i = 0;

void push(char c) {
    stack[++top] = c;
}

void pop() {
    if (top >= 0)
        top--;
}

void printStack() {
    for (int j = 0; j <= top; j++)
        printf("%c", stack[j]);
}

int main() {
    printf("Enter input string (end with $): ");
    scanf("%s", input);

    push('0');

    printf("\nStack\tInput\tAction\n");

    while (1) {
        printf(" ");
```

```
printStack();
printf("\t%s\t", &input[i]);

if (input[i] == 'c') {
    printf("Shift\n");
    push('c');
    i++;
}
else if (input[i] == 'd') {
    printf("Shift\n");
    push('d');
    i++;
}
else if (input[i] == '$') {
    printf("Accept\n");
    break;
}
else {
    printf("Reject\n");
    break;
}
}

return 0;
}
```

10. Test Cases

Valid Input:

ccdc\$

Valid Input:

cd\$

Invalid Input:

ccx\$

11. Parsing Output Format

Example:

Stack	Input	Action
0	ccdc\$	Shift
0c	cdc\$	Shift
0cc	dc\$	Shift
0ccd	c\$	Shift
0ccdc	\$	Accept

Stack	Input	Action
0	ccx\$	Shift
0c	cx\$	Shift
0cc	x\$	Reject

12. Conclusion

In this assignment, an LR(0) parser was successfully constructed. The grammar was augmented, LR(0) item sets were generated, and a parsing table was constructed.

The implementation demonstrates bottom-up parsing using stack operations and DFA-based state transitions.

This assignment strengthens understanding of:

- Compiler parsing techniques
 - LR parsing theory
 - Shift-reduce parsing mechanism
-

13. Screenshots:

```
Ubuntu + - x
Enter input string (end with $): ccdc$

Stack Input Action
0 ccdc$ Shift
0c cdc$ Shift
0cc dc$ Shift
0ccd c$ Shift
0ccdc $ Accept
root@Musfira:/mnt/d/work/Cfiles/Task1# ./A3l10
Enter input string (end with $): cd$

Stack Input Action
0 cd$ Shift
0c d$ Shift
0cd $ Accept
root@Musfira:/mnt/d/work/Cfiles/Task1# c d$
c: command not found
root@Musfira:/mnt/d/work/Cfiles/Task1# c d$
c: command not found
root@Musfira:/mnt/d/work/Cfiles/Task1# ./A3l10
Enter input string (end with $): c d$

Stack Input Action
0 c Shift
0c Reject
root@Musfira:/mnt/d/work/Cfiles/Task1# ./A3l10
Enter input string (end with $): ccx$

Stack Input Action
0 ccx$ Shift
0c cx$ Shift
0cc x$ Reject
root@Musfira:/mnt/d/work/Cfiles/Task1#
```